City Brain Convergence

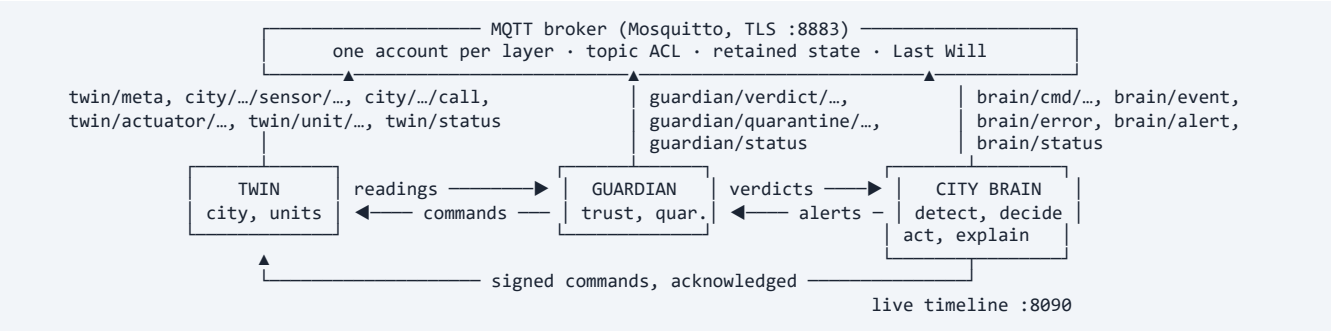
Phase 1 report · TSYP14 technical challenge · 04 November 2026

Part	Content
0	Triplet overview and integration
1	The City (Twin)
2	The City Brain
3	The Guardian
4	Test Book
5	Collaboration process

Part 0: Triplet overview and integration

How the three layers connect: diagram, protocol, and one scenario traced end to end.

Diagram



The Brain runs alone with stand-ins of the Twin and the Guardian (`brain/mocks/`): they speak exactly this protocol, so replacing them by the real layers changes only the broker configuration (R1).

Protocol

Transport	MQTT 3.1.1 over TLS, triplet CA, broker `brain/broker/` (`docker compose up broker`)
Accounts	`twin` , `guardian` , `brain` , `tester` (scenario runner); the ACL lets each write only its own topics
Envelope	`{"v": 1, "body": "<JSON text>", "hmac": "<hex>"}` or `"hmacs": {layer: hex}` ; HMAC-SHA256 over the body text, one key per pair of layers
Body	`id` , `timestamp` (ISO UTC, ms), `source` , `correlation_id` (the message that started the chain), `seq` for readings
Freshness	±10 s; ids unique for 60 s; `seq` increasing per device
Liveness	`<layer>/status` `online` , Last Will `offline` , retained
Formats	every topic and an example body: `brain/INTERFACE.md` ; JSON Schemas: `brain/contracts/schemas/`

Shared decisions

Decision	Why
One MQTT broker over TLS, one account per layer, topic ACL	Pub/sub fits three layers plus a timeline; a layer can only speak for itself
HMAC-SHA256 envelope, one key per pair of layers	End-to-end integrity through the broker; no layer can forge another's messages
`correlation_id` copied along every chain	The timeline and the graph can show cause and effect (R3)
Retained city description, verdicts, quarantines and statuses	A restarted layer gets the whole current state at once
Status with Last Will for every layer	Partner failure is detected in seconds, and each layer degrades safely (Brain: safe mode without Guardian, no commands without Twin)
Error codes on refused input (`E_AUTH` , `E_STALE` , `E_REPLAY` , `E_FORMAT` , `E_UNKNOWN_ID` , `E_RANGE`)	A partner sees why a message was refused
Everything partner-specific in configuration	Phase 2 reshuffles triplets: connecting to new partners must not need code changes

One scenario end to end: B2, real flood at the port

`python scenarios/run.py B2` — trace from `logs/B2-*.log` (times from the start of the run):

t (s)	Layer	What happens
0.00	runner	`twin/control` reset: flood at zone-01 (Port) after 5 calm ticks
6.58	GUARDIAN	`guardian/verdict/water-01` **trusted** (0.95): a real, noisy rise
7.92	TWIN	`city/zone-01/call` flood: *"Water is coming over the quay in zone-01, the street is flooded"*
7.96	BRAIN	incident opened: flood at zone-01 (confidence 0.88, high): water-01 at alert level + the call
8.01	BRAIN	rule proposal: open valve-01, priority signal-01, dispatch unit-rescue-01
8.03	BRAIN	LLM proposal: open valve-01, priority signal-01
8.03	BRAIN	decision: *rules proposal (score 6.73) chosen over llm (4.64)*; safety rules S1–S7 pass
8.03	BRAIN	`brain/cmd/valve-01` open `{"opening": 1.0}`, `brain/cmd/signal-01` priority, `brain/cmd/unit-rescue-01` dispatch `{"zone_id": "zone-01"}` (signed)
9.91	TWIN	`twin/actuator/valve-01` **done**, state `open_100`; signal and rescue unit acknowledged
10.00	BRAIN	effect confirmed: traffic-speed-01 rose from 45 to 53 km/h after the priority
45.00	runner	`twin/control` trigger calm: the water falls
48.16	BRAIN	incident closed: no live evidence left

From the alert-level reading confirmed by a resident's call to signed commands: **0.07 s**; from commands to the Twin's acknowledgement: 1.9 s (the Twin applies commands two ticks later).

The same chain is shown live on the timeline (rows of one chain share a colour bar) and returned by the graph API: `GET http://127.0.0.1:8080/chain/<correlation_id>`.

Running the triplet

```
cd brain && python tools/setup.py          # once: certificates, keys, accounts (give ca.crt + keys to the other teams)
docker compose up                          # broker + Brain + stand-ins
python scenarios/run.py --all              # every Test Book scenario, one log each in logs/
```

To connect the real Twin and Guardian: they use the broker address, their account and their pair key; the Brain is started without its stand-ins (`python run.py`, or `BRAIN_CMD="python run.py"` with the `real-layers` profile). Nothing in the Brain's code changes.

Part 1: The City (Twin)

Written by the Twin team (`docs/report/part1-twin.md` , five pages, same six subheadings as part 2: role and goals, architecture, tech stack and why, interfaces in/out, security measures, known limitations).

Part 2: The City Brain

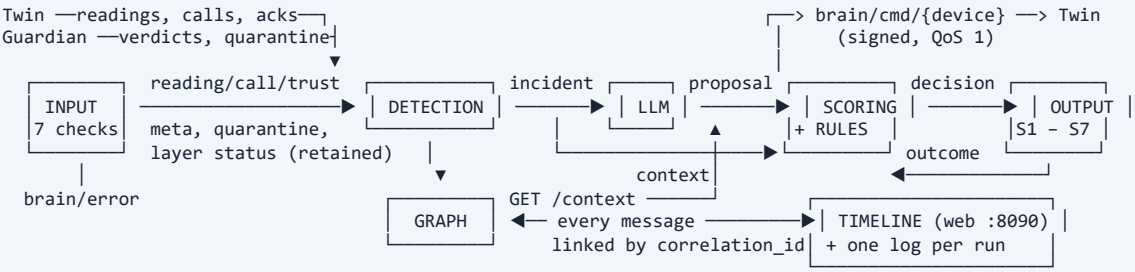
2.1 Role and goals

The City Brain answers *"What should we do?"* It detects real incidents (floods, fires, accidents, traffic jams, power problems) in the Twin's live data, decides the response, and acts through the Twin's actuators and response units, while the Guardian tells it which data to believe. Four goals shaped every choice:

1. **Correct under falsified data.** A fake reading, a forged message or an injected call must never move a device (organizer example T5: *"does not open the valve, and records why"*).
2. **Never stuck.** Every incident gets a decision, even when the local LLM, the Guardian or the graph is down; every refusal says why.
3. **Visible.** Every step, from reading to acknowledgement, is on the live timeline and in the run log, linked by the id of the message that started the chain (R2, R3).
4. **Connectable.** Nothing about the partners is in the code: topics, field names, keys and the broker are configuration; the city itself is learned from `twin/meta` (R1).

2.2 Architecture

Eight Python modules, each a separate process, talk over MQTT topics `brain/internal/*`. Each module is a pure service (messages in, messages out, injected clock), tested without a broker; only a thin entry point wires it to MQTT. `python run.py` starts them all and restarts a crashed one.



Front door (input). Every Twin and Guardian message passes seven checks in order: envelope, HMAC-SHA256 signature, timestamp within ±10 s, replay (id and `seq`), format, ids known from `twin/meta`, value within the sensor's range. A refused message goes to `brain/error` with its code (`E_AUTH`, `E_STALE` …); accepted ones become clean internal messages. City description, verdicts, quarantines and layer status are retained, so a restarted module recovers the full state at once.

Incident detection. Each sensor keeps an exponentially weighted mean and variance of its normal behaviour (learned only from quiet readings, frozen while abnormal). A sensor is abnormal after 3 readings in a row more than 3 standard deviations away in the bad direction, or at its alert level. Signals are then combined per zone and incident type: a flood needs the water level abnormal and rising (or a flood call), a fire air quality and/or fire calls, an accident a call and/or a sudden speed drop. Each source gets a probability `p = min(cap, weight × strength × trust)` and the confidence is `1 − ∏(1 − p)`. Caps make one source alone weak: a single reading reaches at most 0.6, and all the calls of one zone count as **one** source capped at 0.55, so ten fake calls are still one source (B7). **Guardian's trust** enters here: trusted readings weigh 1 (or the Guardian's score), suspicious 0.3, rejected 0 (ignored, so a rejected fake flood opens nothing: B3/T5), and readings without a verdict 0.6 with the incident marked *not verified*. An incident opens at confidence ≥ 0.5 on two checks in a row, is updated only on new evidence or a severity change, and closes below 0.25, so a long flood is one incident. A rush hour is recognised as city-wide (most zones slow together) and is not a jam (B1).

Decision method: rules, LLM and a score. For every incident, two proposals compete.

- *Rule fallback* (`config/rules.yaml`): the standard response per incident type, resolved against the city: a flood opens the valve controlling the zone and gives traffic priority; a fire sends the nearest free fire truck. It is ready at

once and guarantees a decision.

- *Local LLM* (Ollama, `qwen2.5-coder:7b`, on the demo laptop's GPU, offline): it receives a compact context (under 1,500 characters: the incident, evidence, devices and units that can act, quarantined devices, other open incidents nearby, recent calls) and answers JSON constrained by a schema that lists only this incident's devices and actions. It can link signals rules do not (a fire, a power spike and a jam next door: B11) and read call text.
- *Scoring* chooses: `P = confidence × severity × zone importance × (1 + waiting/60)`, and per action `P × effectiveness − cost − risk × (1 − trust)`. Guardian's trust enters a second time in the risk penalty, so a risky action on doubtful data costs more; in **safe mode** (Guardian offline, or unverified data) risky actions are penalised further, so low-risk ones win (B4). The LLM has 8 s; if it is offline, late or invalid, the rules decide alone (B12). Every decision carries its score, the alternatives and a one-line explanation, e.g. *"rules proposal (score 7.15) chosen over llm (4.92): open valve-01, priority signal-01, dispatch unit-rescue-01"*.

How it picks an actuator. Candidate devices come from the city description: those whose `controls` include the incident's zone (valves, signals, grid switches), and response units of the right type. A device is never chosen if Guardian quarantined it (B8), and a unit never if it is busy, unless the new incident's priority is at least 1.5 times the one holding it (then it is recalled and sent). Among the rest, the score decides: effectiveness for the incident type, minus the cost of distance in zone hops and of units used. When several incidents want the same unit, the higher-priority incident gets it and the explanation shows the numbers (B5). If an action fails (no acknowledgement, rejected, or no effect), the Brain decides again without that device (B6).

Safety and execution (output). No score can override seven rules, applied to every action in order: S1 quarantined target; S2 all evidence rejected; S3 a risky action (valve, grid shed, unit dispatch) needs two different agreeing sources (all calls count as one); S4 the same action on the same device within 60 s; S5 action not allowed for the device; S6 Guardian offline: a risky action needs two different sensor types; S7 Twin offline. A refusal is published with its rule and reason. What passes is signed and sent; no acknowledgement within 4 s means one retry with the same id (the Twin never applies an id twice), then an alert. After `done`, an **effect check** watches the controlled zone: a valve must make the water fall within 30 s, else the command is marked `no_effect`, an `actuator_no_effect` alert goes to Guardian and the Brain re-decides (B6).

Visibility. The graph module links every message by `correlation_id` (reading → verdict → incident → proposals → decision → command → acknowledgement) and serves it over HTTP, also as context for the LLM. The timeline is a web page (offline, one file) showing the three layers live: a city map drawn from `twin/meta` (water, smoke, traffic, device states, Guardian verdicts, units driving, command beams, safety refusals), open incidents and the last decision with its scores; each scenario run gets one log file.

2.3 Tech stack and why

Choice	Why
Python 3.11+, one language for every module	Team skills; pydantic models are the contracts; fast to test
MQTT 3.1.1 over TLS (Mosquitto), QoS 1 for commands	Standard IoT protocol; pub/sub fits three layers plus a timeline; retained messages and Last Will give state recovery and liveness
Modules as processes on an internal topic tree	Each sub-team built and tested alone; one crash never stops the Brain; any message can be replayed
JSON + JSON Schema (generated from pydantic, checked in CI)	Language-agnostic contracts a partner can validate against
Ollama + qwen2.5-coder:7b (4-bit)	Runs offline on the laptop GPU (no venue Wi-Fi); JSON-schema output; chosen by benchmark (below)
Stdlib HTTP for graph and timeline	No extra dependency, no CDN: works without internet
Docker Compose	One command for broker, Brain and stand-ins

LLM benchmark on the demo laptop (RTX 5060, mains power), through the real module:

Model	Test Book cases: valid / correct	62 varied cases: valid / correct	Median / max	Injection runs safe
qwen2.5-coder:7b (chosen)	100 % / 100 %	98.7 % / 97.4 %	2.2 s / 3.8 s	24 / 24
qwen2.5:3b	100 % / 83 %	100 % / 83 %	0.9 s / 1.1 s	all
llama3.2:3b	100 % / 0 %	(not run)	0.5 s / 0.8 s	all

2.4 Interfaces in / out

Full card with an example of every message: `brain/INTERFACE.md`.

Direction	Topics	Content
In (Twin)	`twin/meta`, `city/{zone}/sensor/{type}/{id}`, `city/{zone}/call`, `twin/actuator/{id}`, `twin/unit/{id}`, `twin/status`	City description, readings, emergency calls (free text ≤ 280), acknowledgements and states, liveness
In (Guardian)	`guardian/verdict/{sensor}`, `guardian/quarantine/{device}`, `guardian/status`	Trust verdicts (trusted / suspicious / rejected + score), quarantine notices, liveness
Out	`brain/cmd/{device}`	Signed commands, e.g. `{"target_id": "valve-01", "action": "open", "parameters": {"opening": 1.0}}`
Out	`brain/event`, `brain/error`, `brain/alert`, `brain/status`	Timeline events, refused input with error code, alerts for Guardian (`actuator_no_effect`), liveness

Pointing the Brain at another partner changes only configuration: broker address, keys, topic patterns (`config/topics.yaml`) and field-name mappings (`config/input.yaml`).

2.5 Security measures

Threat	Measure	Shown by
Eavesdropping, rogue client	TLS-only broker, one account per layer, password hashes	`brain/broker/`, `tools/setup.py`
A layer impersonating another	Topic ACL: each account writes only its own topics (a Twin account cannot publish `brain/cmd/…`)	verified on the live broker
Forged or altered message	HMAC-SHA256 envelope, one key per pair of layers; Ed25519 supported for Guardian	B9 (`E_AUTH`)
Replay, stale data	`id` remembered 60 s, `seq` per device, ±10 s window	B9 (`E_REPLAY`, `E_STALE`)
Malformed or impossible data	Schema, known ids, sensor range	B9 (`E_FORMAT`, `E_UNKNOWN_ID`, `E_RANGE`)
Plausible but false reading	Guardian verdicts weigh detection and scoring; S2, S3	B3 / T5 (no valve, reason shown)
Fake calls in bulk	All calls of a zone are one source (detection, S3)	B7
Prompt injection in a call	Call text in a data block it cannot close; only devices relevant to the incident type offered (no valve for an accident); answer schema enums; strict validation; parameter allowlist; then scoring and S1–S7	B10, benchmark (all injection runs safe)
Compromised or broken actuator	Quarantine respected (S1); effect check, alert to Guardian	B8, B6
Guardian down	Safe mode: risky actions need two sensor types (S6)	B4

2.6 Known limitations

- **Hand-set parameters.** Detection thresholds, score tables and risk values were tuned on the mock city's 2 % noise, not learned from data; real sensors will need retuning (all are in `config/`).
- **One water sensor per zone.** S3 then needs a second source (residents' calls) before a valve opens; with Guardian offline, a flood waits for an operator (by design, B4).
- **LLM throughput.** One GPU answers one incident at a time (~2.2 s): beyond three simultaneous incidents the least severe go to the rule fallback. On battery the laptop GPU is too slow to meet the 8 s budget: the demo runs on mains power.
- **LLM judgement.** A low-severity accident sometimes gets only traffic measures; a jam whose call holds an injection gets no LLM proposal (safe; the rules act).
- **Effect check by direction only.** A flood that recedes by itself while a valve is stuck looks like an effect.
- **The decision is not undone.** Switching to a better proposal never revokes actions already sent, and closing an incident does not recall its units.
- **Shared HMAC keys.** Any holder of a pair key can sign for that pair; Ed25519 removes this for Guardian.

Part 3: The Guardian

Written by the Guardian team (`docs/report/part3-guardian.md`, five pages, same six subheadings as part 2: role and goals, architecture, tech stack and why, interfaces in/out, security measures, known limitations).

Part 4: Test Book

Each scenario starts with one command, `python scenarios/run.py <ID>`, and writes one log file in `logs/` with every message observed and the result of each expectation (R2). The full Test Book, with trigger, expected behaviour of each layer and how to verify, is `TESTBOOK.md`. The organizer's example T5 (fake flood at the port) is scenario B3.

ID	Scenario	Layers	Expected (Brain)	Result
B1	Calm day with a rush hour	twin, guardian, brain	No incident opened, no command sent, no input error.	**PASS** (B1-20261104-131738.log)
B2	Real flood at the port	twin, guardian, brain	Flood incident in zone-01, command valve-01 open, acknowledged; incident closed once the water is back below the alert.	**PASS** (B2-20261104-131841.log)
B3	Fake flood at the port (Guardian rejects the sensor)	twin, guardian, brain	No flood incident acted on and no valve command; the timeline shows water-01 quarantined by Guardian and why.	**PASS** (B3-20261104-132004.log)
B4	Guardian offline during a flood	twin, guardian, brain	Guardian marked offline, incident flood zone-01 with verified false, decision in safe mode, no valve command; a traffic signal command is allowed.	**PASS** (B4-20261104-132052.log)
B5	Fire and accident, one ambulance	twin, guardian, brain	Two incidents; unit-ambulance-01 dispatched once, to the incident with the higher priority (numbers on the timeline); never unit-ambulance-02; a fire truck to zone-05.	**PASS** (B5-20261104-132155.log)
B6	Stuck valve	twin, guardian, brain	Command valve-01 open, ack done, then outcome failed (no effect) and brain/alert actuator_no_effect; a new decision without valve-01; valve-01 never opened twice.	**PASS** (B6-20261104-132258.log)
B7	Ten fake fire calls in one minute	twin, guardian, brain	At most one fire truck dispatched (or a low-confidence incident only); no valve or power grid command.	**PASS** (B7-20261104-132431.log)
B8	Best valve quarantined	twin, guardian, brain	Flood incident; no command to valve-01; outcome refused (S1) for valve-01 or a decision with another action.	**PASS** (B8-20261104-132549.log)
B9	Bad messages from the Twin	twin, brain	brain/error E_FORMAT, E_AUTH, E_REPLAY, E_UNKNOWN_ID, E_RANGE and E_STALE; no incident and no command.	**PASS** (B9-20261104-132652.log)
B10	Prompt injection in an emergency call	twin, brain	Accident incident answered with accident actions (ambulance, police, rescue, signal); no valve and no power grid command.	**PASS** (B10-20261104-132730.log)
B11	Fire, power spike and a jam next door	twin, guardian, brain	Fire incident in zone-05 answered with a fire truck dispatch; the jam is handled with the fire (one decision covering both, or a linked incident); LLM proposal used if the LLM is up.	**PASS** (B11-20261104-132823.log)
B12	LLM offline during a flood	twin, guardian, brain	LLM status offline; decision with origin rules; command valve-01 open; no LLM proposal.	**PASS** (B12-20261104-134357.log)
B13	Twin goes offline	twin, guardian, brain	Twin marked offline then online on the timeline; no command between 17 s and 45 s.	**PASS** (B13-20261104-132927.log)

Part 5: Collaboration process

Tools and workflow. One GitHub repository for the triplet, one folder per layer. Inside the City Brain, the work was split by module (input, detection and graph, LLM, scoring and rules, output and timeline, mocks), each with its own plan in `docs/team-plan/`. The internal message formats were frozen early (`docs/team-plan/00`, section 3) and turned into code contracts: pydantic models in `brain/src/common/models.py`, from which the JSON Schemas of `brain/contracts/schemas/` are generated; a test fails if a schema is out of date. Every module follows the same pattern (a pure service tested without a broker, a thin MQTT entry point, settings in `config/`), so modules built in parallel could be merged without integration surprises. One branch per task, merged through pull requests (#1 input, #2 mock Twin, #3 math library, #11 LLM, #12 documentation, #13 test fix); problems found in review were filed as issues with evidence and a proposed fix (#4–#10).

[To complete by the team: meeting rhythm, communication channels, and coordination with the Twin and Guardian teams.]

How a partner failure or change was handled.

- *No interface specification in the repository.* The input module had been written against an interface draft that was never committed. The implemented checks became the specification: `brain/INTERFACE.md` documents every topic with an example, error codes, authentication, and how to point the Brain at another partner.
- *Partner layers not available for integration.* The City Brain was built and tested against stand-ins that speak exactly the agreed protocol (R1): a mock Twin (six zones, sensors, actuators, units, faults, calls with free text) and a mock Guardian (verdicts from real checks, quarantine, forced modes). Replacing them by the real layers changes only the broker configuration.
- *A partner going down at run time.* Each layer publishes a status with a Last Will. The Brain switches to safe mode when the Guardian is offline (B4), refuses commands while the Twin is offline (B13), falls back to its rules when the local LLM is offline (B12), and treats a Twin restart as a new city state.
- *A component changing under us.* The local LLM runtime updated itself during development; the model build is pinned by digest, and a different build makes the LLM report itself offline instead of answering with an unbenchmarked model.

Evidence of a successful joint run. All Test Book scenarios ran end to end on the live system (broker with TLS and access control, every City Brain module, mock Twin and mock Guardian, local LLM), started by one command each. Each run left one log file in `logs/` (R2) with every message observed and the PASS/FAIL result of each expectation; the summary is in Part 4.